Estrutura do Sistema — Gestão de Laboratórios (DLab)

Sistema de gestão de ocupação de laboratórios, submissão e aprovação de atividades (aulas, visitas, projetos, estágios), gestão de inventário por movimentações e relatórios de utilização.

ÍNDICE

1. Requisitos Funcionais
2. Fluxos de Atividades
3. Front — React
    - 3.1. Estrutura de Arquivos
    - 3.2. Wireframe / Storyboard — Tela por Tela
    - 3.3. Services
4. Backend — Node.js
    - 4.1. Estrutura de Arquivos
    - 4.2. BD
    - 4.3. Models
    - 4.4. DTOs
    - 4.5. Endpoints

1. REQUISITOS FUNCIONAIS

1. Gestão de Utilizadores e Autenticação

- **RF01 – Registo e Login**: O sistema deve permitir o login de utilizadores através de email e senha_hash.
- **RF02 – Controlo de Acesso (RBAC)**: O sistema deve validar as permissões das rotas com base no utilizador_tipo (Admin, Professor, Técnico, Coordenador DLab, Supervisor, Chefe de Departamento).

2. Submissão de Propostas (Aulas, Visitas, Projetos e Estágios)

- **RF03 – Criação de Atividade**: O sistema deve permitir que se submeta uma proposta de actividade indicando o laboratório e o tipo.
- **RF04 – Proposta de Calendário**: O utilizador deve poder indicar um ou múltiplos blocos de datas/horas (agendamentos) no mesmo formulário de submissão da atividade.
- **RF05 – Detalhes de Aula**: Se o tipo for "Aula", o sistema deve exigir o vínculo a uma disciplina do curso (curso_disciplina_id), o número de participantes e se precisa de apoio técnico.
- **RF06 – Detalhes de Projeto/Estágio**: Se o tipo for "Projeto" ou "Estágio", o sistema deve exigir o upload de um ficheiro (anexo_path, que será submetido após a conclusão da actividade), um professor responsável e as datas macro de início e fim do contrato.
- **RF07 – Vínculo de Aluno**: No caso de um Estágio, o sistema deve permitir associar um estudante matriculado (estudante_id).
- **RF08 – Detalhes de Visita**: Se o tipo de atividade for "Visita", o sistema deve exigir obrigatoriamente o registo dos dados do visitante principal (nome_visitante), a instituição de origem, o telefone e o e-mail de contacto.

3. Fluxo de Aprovação em Duas Etapas

- **RF09 – Validação do DLab**: O Coordenador do DLab deve poder visualizar as atividades em estado = 'pendente', registar o seu voto (aprovacao_decisao) e um comentário. O estado da atividade deve mudar para aprovado_dlab.
- **RF10 – Validação do Supervisor**: O Supervisor deve poder visualizar apenas as atividades que já foram aprovadas pelo DLab. Ao emitir o voto favorável, o estado da atividade muda para aprovado_supervisor.
- **RF11 – Rejeição de Atividade**: Se o DLab ou o Supervisor rejeitarem a proposta, a atividade muda imediatamente para rejeitado e o fluxo é interrompido.

4. Calendário e Ocupação do Laboratório

- **RF12 – Visualização do Calendário**: O sistema deve exibir um calendário mensal de ocupação por laboratório, listando apenas os agendamentos cujas atividades mães estejam em estado aprovado_supervisor.
- **RF13 – Bloqueio de Choques**: O sistema deve impedir a gravação final ou alertar os aprovadores caso dois agendamentos aprovados coincidam no mesmo laboratório, data e intervalo de horas.
- **RF14 – Confirmação de Presença**: O Utilizador (Admin, professor, técnico, etc) deve poder marcar um agendamento como realizado = true após a utilização efetiva do espaço. Isso deverá disparar uma mensagem para o técnico para que ele confirme o término da actividade. (primeiro o professor e depois o técnico)

5. Gestão de Inventário (Materiais)

- **RF15 – Inventário por Laboratório**: O sistema deve listar o inventário base de cada laboratório, colhendo o nome do material, a categoria, a quantidade em stock, a unidade de medida, o estado de conservação e o alerta de stock mínimo.
- **RF16 – Registo de Movimentação de Stock (Histórico)**: Toda e qualquer alteração na quantidade de um material deve gerar um registo obrigatório de movimentação. Esse registo deve colher quem fez a alteração, o dia e a hora, a quantidade adicionada (valores positivos) ou removida (valores negativos), o motivo padronizado e uma descrição detalhada de justificação.
- **RF17 – Bloqueio de Edição Direta**: Não deve ser permitido alterar o campo de quantidade diretamente na tabela principal de materiais. O stock disponível deve ser sempre o resultado da soma de todas as movimentações registadas no histórico daquele material.
- **RF18 – Alerta de Stock Mínimo**: O sistema deve emitir um alerta visual no painel do Técnico sempre que a quantidade atual de um material for igual ou inferior à quantidade_minima.

6. Relatórios Dinâmicos

- **RF19 – Geração de Relatório Mensal**: O sistema deve permitir que um Administrador ou Técnico gere um relatório de fecho de mês contendo o histórico e a contagem de atividades realizadas (COUNT de agendamentos reais) exportado em formato estruturado (dados_json).

Nota sobre Técnico vs Assistente (esclarecimento aplicado ao modelo, sem renumerar os RF acima):

- **Toda** atividade precisa de um **Técnico** (papel validador) — é quem confirma em segundo lugar, depois do professor/primeiro utilizador, a conclusão da actividade (RF14).
- **Nem toda** atividade precisa de **Assistente** — pessoa extra (indicada pelo professor/aprovador) para ajudar na atividade.
- Ambos os papéis são ligados à atividade pela tabela actividade_tecnico (tecnico_tipo: validador | assistente).
- O campo actividades.precisa_assistente existe porque a atividade nasce como **proposta**: informa a quem aprova se deve ou não atribuir um assistente.

2. FLUXOS DE ATIVIDADES

Passo 1 — Registo e Gestão de Perfis

Para começar a usar a plataforma, qualquer pessoa da universidade tem de criar uma conta.

- **Registo de Utilizadores**: recolhe-se o nome completo, e-mail institucional, palavra-passe de acesso e o curso (no caso dos estudantes).
- **Gestão de Perfis**: os utilizadores comuns podem editar os seus próprios dados de cadastro (como nome ou palavra-passe) a qualquer momento.
- A alteração do cargo ou nível de acesso (se a pessoa passa de Professor a Coordenador ou Supervisor, por exemplo) é uma ação exclusiva que só pode ser feita manualmente pelo **Administrador** do sistema.

Passo 2 — Cadastro da Infraestrutura e Recursos (Administração)

A administração configura a base do sistema antes de qualquer pedido acontecer.

- **Laboratórios e Cursos**: regista-se o nome e especialidade de cada laboratório, bem como os cursos e o catálogo de disciplinas com os seus respetivos semestres.
- **Materiais**: regista-se o inventário inicial de cada laboratório. Colhe-se o nome do material, a categoria (equipamento, composto, vidraria ou consumível), a quantidade inicial em armazém, a unidade de medida (ex: gramas, unidades, litros), o estado (disponível, em uso, manutenção ou esgotado) e o alerta de stock mínimo.
- O **stock inicial** entra como a **primeira movimentação** no histórico (motivo = compra_stock ou ajuste_inventario), respeitando o RF17 (a quantidade nasce do histórico, nunca é editada diretamente).

Passo 3 — Submissão da Proposta com Pedido de Materiais

O utilizador preenche o formulário único indicando o laboratório, observações e anexando todos os dias e horários específicos que pretende ocupar.

Dependendo da finalidade, ele detalha:

- **Aula**: disciplina, número de alunos.

- **Visita**: nome do visitante responsável, instituição, telefone, e-mail.

- **Projeto / Estágio**: título, descrição, datas macro do contrato, documento em PDF, nome do aluno (apenas para estágios).

- **Pedido de Materiais (opcional)**: no mesmo formulário, o professor pode declarar os consumíveis ou equipamentos que espera utilizar. O sistema exibe apenas os materiais que pertencem ao laboratório escolhido. O professor seleciona os itens e indica a quantidade estimada que cada sessão vai consumir.

Passo 4 — Fluxo de Validação (DLab e Supervisor)

A proposta contendo os detalhes da atividade, as datas propostas e a lista de materiais solicitados passa pelas duas etapas de avaliação:

- **Validação pelo Coordenador do DLab**: analisa o pedido completo e emite uma decisão com um parecer técnico por escrito. Se aprovado, o pedido segue para a fase seguinte.
- **Validação pelo Supervisor**: faz a avaliação final de liderança sobre todo o pacote. Ao emitir o voto favorável de "Aprovação Final", o sistema liberta as datas. Nesta etapa é atribuído o **Técnico (validador) obrigatório** e, se precisa_assistente, um **Assistente**.

Passo 5 — Impacto no Calendário e Baixa de Materiais

Após a aprovação final do Supervisor, o sistema executa as seguintes ações de negócio:

- **Bloqueio de Agenda**: os horários solicitados ficam marcados como "Ocupados" no calendário público da universidade para evitar choques com outros professores.
- **Conclusão da Atividade**: no dia do evento, após a utilização real do espaço, o utilizador (professor) entra no sistema e marca aquele horário específico como "Concluído/Realizado", após isso, o técnico (validador) deve fazer o mesmo para concluir definitivamente a actividade. Nesse exato momento, o sistema faz a redução automática das quantidades no inventário daquele laboratório, subtraindo o stock que o professor tinha pedido no formulário (historico_materiais).

Passo 6 — Ajustes Manuais de Inventário (Fora de Aula)

Para garantir que o inventário real corresponde sempre ao inventário do sistema, o técnico ou administrador tem autonomia para intervir diretamente no stock a qualquer momento.

Se um componente se partir, se um composto químico for derramado por acidente fora do horário de aula, ou se a universidade comprar novos materiais, o técnico abre o painel de inventário e faz a adição ou subtração manual da quantidade de forma imediata, justificando a alteração. O sistema deteta automaticamente quem está a fazer a alteração (o e-mail do técnico) e guarda o dia e a hora. O técnico insere se está a adicionar stock (ex: +10 por compra) ou a retirar stock (ex: -2 por quebra), deixa o campo de atividade em branco (já que aconteceu fora das aulas), seleciona o motivo correto num menu de opções e escreve obrigatoriamente uma justificação detalhada sobre o sucedido. O stock atualizado do laboratório passa a refletir este ajuste.

Passo 7 — Relatórios de Utilização (Automáticos ou Manuais)

Para fins estatísticos e de prestação de contas, a plataforma compila o resumo de tudo o que foi realizado nos laboratórios.

- **Relatório Automático**: no final de cada mês, o sistema gera de forma automática um resumo estatístico fechado com a contagem de atividades e uso daquele período.
- **Relatório Manual**: a qualquer momento, um utilizador autorizado pode aceder à área de relatórios e fazer uma extração personalizada. Ele insere manualmente qual é o mês, o ano e o departamento específico que deseja analisar, e o sistema gera o documento detalhado a pedido.

3. FRONT — REACT

Stack Frontend

- **react** + **vite**
- **tailwindcss** + @tailwindcss/forms + @tailwindcss/typography
- **shadcn/ui** (componentes base sobre Tailwind)
- **react-hook-form** + **zod** (formulários + validação, partilhável com o back)
- **@tanstack/react-query** (cache e fetching)
- **zustand** (estado global leve: auth, notificações)
- **react-router-dom** (routing)
- **@tanstack/react-table** (tabelas de stock/listagens)
- **recharts** (gráficos dos relatórios)
- **date-fns** (datas)
- **sonner** (toasts)
- **lucide-react** (ícones)
- **socket.io-client** (alertas de stock em tempo real)

Convenção de Roles

A = Admin | P = Professor | T = Técnico | C = Coordenador DLab | S = Supervisor | CD = Chefe de Departamento

3.1. ESTRUTURA DE ARQUIVOS

```
src/
├── assets/
│   ├── icons/
│   └── images/
│
├── components/
│   ├── ui/
│   │   ├── Button.tsx
│   │   ├── Input.tsx
│   │   ├── Select.tsx
│   │   ├── Textarea.tsx
│   │   ├── Badge.tsx              # Estado de actividade, tipo, role
│   │   ├── Card.tsx
│   │   ├── Spinner.tsx
│   │   ├── Alert.tsx
│   │   ├── Tooltip.tsx
│   │   ├── Modal.tsx              # Base modal com overlay e portal
│   │   ├── ConfirmDialog.tsx        # Modal de confirmação de ação destrutiva
│   │   ├── EmptyState.tsx           # Ecrã vazio com ícone e CTA
│   │   ├── FileUpload.tsx
│   │   ├── DateTimePicker.tsx
│   │   ├── DateRangePicker.tsx
│   │   ├── StockAlertBadge.tsx       # Badge vermelho: quantidade ≤ mínimo
│   │   ├── Table.tsx              # Tabela genérica: colunas + dados + loading
│   │   ├── FiltersTable.tsx         # Barra de filtros genérica (inputs + selects + reset)
│   │   └── Pagination.tsx
│   │
```

```
│   ├── layout/
│   │   ├── AppShell.tsx            # Sidebar + Topbar + área de conteúdo
│   │   ├── Sidebar.tsx             # Links filtrados por role
│   │   ├── Topbar.tsx              # Utilizador, badge de alertas, logout
│   │   ├── PageHeader.tsx          # Título + breadcrumb + ação primária
│   │   ├── AuthLayout.tsx
│   │   ├── Form.tsx
│   │   ├── Modal.tsx
│   │   ├── TopPageButton.tsx       # Scroll para o topo da pagina
│   │   └── ProtectedRoute.tsx      # Guard de rota por role
│   │
│   ├── modal/
│   │   ├── CursoUpsertModal.tsx
│   │   ├── ActividadeUpsertModal.tsx       # Formulário multistep
│   │   ├── MaterialUpsertModal.tsx
│   │   ├── HistoricoMaterialUpsertModal.tsx
│   │   ├── LaboratorioUpsertModal.tsx
│   │   ├── DisciplinasUpsertModal.tsx
│   │   ├── UtilizadorUpsertModal.tsx       # Cria e edita (modo determinado pelo param :id)
│   │   ├── EstudanteUpsertModal.tsx        # Cria e edita (modo determinado pelo param :id)
│   │   ├── ActividadeTecnicoUpsert.tsx
│   │   └── RelatorioUpsertModal.tsx
│   │
│   └── dashboard/
│       ├── StatCard.tsx
│       ├── ActividadesPendentesWidget.tsx
│       ├── StockAlertsWidget.tsx
│       └── ProximasAulasWidget.tsx
│
├── pages/
│   ├── auth/
│   │   └── Login.tsx               # link / — acesso A,P,T,C,S,CD
│   ├── dashboard/
│   │   └── Dashboard.tsx           # link /inicio — acesso A,P,T,C,S,CD
│   ├── utilizadores/
│   │   ├── Perfil.tsx              # link /perfil
│   │   ├── UtilizadoresList.tsx    # link /users — acesso CRUD: A
│   ├── laboratorios/
│   │   ├── LaboratoriosList.tsx    # link /labs — acesso CRUD: A — acesso R: P,T,C,S,CD
│   │   ├── LaboratorioDetalhe.tsx  # link /labs/{id} — acesso R: A,P,T,C,S,CD
│   ├── cursos/
│   │   ├── CursosList.tsx          # link /cursos — acesso CRUD: A — acesso R: P,T,C,S,CD
│   │   ├── DisciplinasList.tsx     # link /disciplinas — acesso CRUD: A — acesso R: P,T,C,S,CD
│   ├── estudantes/
│   │   ├── EstudantesList.tsx      # link /estudantes — acesso CRUD: A,P,C,S,CD — acesso R: T
│   ├── actividades/
│   │   ├── ActividadesList.tsx     # link /actividades — acesso CRUD: A,P,C,S,CD — acesso R: T
│   │   └── ActividadeDetalhe.tsx   # link /actividades/{id}
│   ├── aprovacoes/
│   │   ├── AprovacoesList.tsx      # link /aprovacoes — acesso CRUD: A,C,S — acesso R: CD   (Fila
adaptativa: DLab (pendente) ou Supervisor (aprovado_dlab))
│   │   └── AprovacaoDetalhe.tsx    # link /aprovacoes/{id}
```

```
│   ├── calendario/
│   │   └── Calendario.tsx              # link /calendario — apresenta as actividades; ao clicar abre a página da
mesma
│   ├── materiais/
│   │   ├── MateriaisList.tsx           # link /materiais — acesso CRUD: A,T,C,S,CD — acesso R: P
│   │   ├── MateriaisHistoricoList.tsx  # link /materiais/historico — acesso CRUD: A,T,S,CD
│   │   └── MaterialDetalhe.tsx         # link /materiais/{id} — apresenta a informação do material
selecionado, inclusive o seu histórico
│   └── relatorios/
│       ├── RelatoriosList.tsx          # link /relatorios — acesso CRUD: A,T — acesso R: C,S,CD
│       ├── RelatorioDetalhe.tsx        # link /relatorios/{id} — acesso R: A,T,C,S,CD
│
├── hooks/
│   ├── useAuth.ts
│   ├── useActividades.ts
│   ├── useMateriais.ts
│   ├── useAprovacoes.ts
│   ├── useCalendario.ts
│   └── useRelatorios.ts
│
├── services/
│   ├── api.ts                  # Instância axios com interceptor JWT
│   ├── enums.ts
│   ├── auth.service.ts
│   ├── utilizadores.service.ts
│   ├── laboratorios.service.ts
│   ├── cursos.service.ts
│   ├── actividades.service.ts
│   ├── aprovacoes.service.ts
│   ├── agendamentos.service.ts
│   ├── materiais.service.ts
│   ├── movimentacoes.service.ts
│   └── relatorios.service.ts
│
├── types/
│   ├── auth.types.ts
│   ├── utilizador.types.ts
│   ├── estudantes.types.ts
│   ├── laboratorio.types.ts
│   ├── curso.types.ts
│   ├── disciplina.types.ts
│   ├── actividade.types.ts
│   ├── aprovacao.types.ts
│   ├── agendamento.types.ts
│   ├── material.types.ts
│   ├── movimentacao.types.ts
│   └── relatorio.types.ts
│
├── utils/
│   ├── roleGuard.ts
│   ├── formatDate.ts
│   ├── formatEstado.ts         # → label + cor de Badge
```

```
│   └── constants.ts
│
├── context/
│   ├── AuthContext.tsx
│   └── NotificacoesContext.tsx       # Alertas de stock em tempo real
│
├── router/
│   └── AppRouter.tsx
│
└── main.tsx
```

3.2. WIREFRAME / STORYBOARD — TELA POR TELA

Legenda de estados: **loading** (dados a carregar) · **vazio** (sem registos) · **erro** (falha de rede/validação). Cada tela indica o layout (regiões), os componentes usados e os estados.

Login — / · acesso: A,P,T,C,S,CD

Página: pages/auth/Login.tsx · Layout: AuthLayout

```
┌──────────────────────────────────────┐
│          [LOGO]                      │
│   Gestão de Laboratórios             │
│                                      │
│  Email   [_______________]           │
│  Senha   [_______________]           │
│                                      │
│        [  Entrar  ]                  │
│                                      │
│  ⚠ Credenciais inválidas             │
└──────────────────────────────────────┘
```

- **Componentes**: Input (email, senha), Button (Entrar), Alert (erro de credenciais), Spinner (no submit).
- **Estados**: loading → botão desativado + Spinner; erro → Alert vermelho. Sucesso → redireciona para /inicio.

Dashboard — /inicio · acesso: A,P,T,C,S,CD

Página: pages/dashboard/Dashboard.tsx · Layout: AppShell (Sidebar + Topbar) + PageHeader

```
┌── Sidebar ──┬──────────── Topbar (user · 🔔3 · logout) ────────┐
│ ▸ Início    │ Bom dia, [Nome]                                  │
│ ▸ Activid.  │ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ │
│ ▸ Aprovaç.  │ │Pendent.│ │Aprovad.│ │Stock ⚠ │ │Hoje    │ │
│ ▸ Calendár. │ │   4    │ │   12   │ │   3    │ │   2    │ │
│ ▸ Materiais │ └──────────┘ └──────────┘ └──────────┘ └──────────┘ │
│ ▸ Relatór.  │ ┌──────────────────────┐ ┌──────────────────────────┐ │
```

```
│ ▸ Labs      │ │ Actividades      │ │ Alertas de Stock    │ │
│ ▸ Cursos    │ │ Pendentes (C/S)  │ │ (T)                 │ │
│ ▸ Estudant. │ └──────────────────┘ └─────────────────────┘ │
│ ▸ Utilizad. │ ┌──────────────────────────────────────────┐ │
│             │ │ Próximas Aulas / Agendamentos aprovados  │ │
│             │ └──────────────────────────────────────────┘ │
└─────────────┴──────────────────────────────────────────────┘
```

- **Componentes**: StatCard ×4 (pendentes, aprovadas, alertas de stock, atividades de hoje); ActividadesPendentesWidget (fila de aprovação para C/S); StockAlertsWidget (visível a T); ProximasAulasWidget (agendamentos aprovados próximos).
- **Sidebar**: links filtrados por role via roleGuard.
- **Topbar**: identificação do utilizador, badge de alertas de stock (via NotificacoesContext), logout.
- **Estados**: loading → skeletons nos cards; vazio → EmptyState por widget.

Perfil — /perfil · acesso: todos (o próprio)

Página: pages/utilizadores/Perfil.tsx · Layout: AppShell + PageHeader + Form

```
┌─ Meu Perfil ──────────────────────────────┐
│ Nome    [____________________]            │
│ Email   [____________________] (locked)   │
│ Tipo    [ Professor ] (read-only)         │
│ ── Alterar senha ──                       │
│ Nova senha    [___________]               │
│ Confirmar     [___________]               │
│                [ Guardar ]                │
└───────────────────────────────────────────┘
```

- **Componentes**: Input (nome, nova senha, confirmar), Badge (tipo — read-only), Button (Guardar) + ConfirmDialog.
- **Regra**: o campo tipo (cargo/nível de acesso) só é editável por Admin em /users (Passo 1 do fluxo). O utilizador comum altera apenas nome e senha.
- **Estados**: loading (dados do perfil), erro (validação de senha não coincide via Alert).

Utilizadores — /users · acesso CRUD: A

Página: pages/utilizadores/UtilizadoresList.tsx · Layout: AppShell + PageHeader + FiltersTable + Table + Pagination

```
┌─ Utilizadores ──────────────────────── [+ Novo] ┐
│ 🔍[nome] [email] [tipo ▾]      [Reset]          │
│ ┌──────────────────────────────────────────────┐ │
│ │ Nome     │ Email      │ Tipo   │ ···  │      │
│ │ João S.  │ j@isptec.. │ Prof.  │ ✎ 🔑 │      │
│ │ Ana M.   │ a@isptec.. │ Técn.  │ ✎ 🔑 │      │
│ └──────────────────────────────────────────────┘ │
```

```
│                    [◂ 1 2 3 ▸]     │
└──────────────────────────────────────────────────────┘
```

- **Componentes**: FiltersTable (nome, email, tipo), Table, Badge (tipo). Ações de linha: editar (UtilizadorUpsertModal), reset password (🔑 → envia ResetPassword). Pagination.
- **Ação primária**: "Novo" → UtilizadorUpsertModal (cria; modo determinado pela ausência de :id).
- **Estados**: loading → linhas skeleton; vazio → EmptyState ("Sem utilizadores"); erro → Alert.

Laboratórios — /labs · acesso CRUD: A · R: P,T,C,S,CD

Página: pages/laboratorios/LaboratoriosList.tsx · Layout: AppShell + PageHeader + Table

```
┌─ Laboratórios ──────────────────────────── [+ Novo] ┐
│ Nome          │ Tipo    │ Descrição │ ··· │
│ Lab. Química 1 │ Química │ ...       │ → │
│ Lab. Física    │ Física  │ ...       │ → │
└──────────────────────────────────────────────────────┘
```

- **Componentes**: Table (nome, tipo, descrição), Badge (tipo). "Novo" → LaboratorioUpsertModal.
- **Navegação**: linha → /labs/{id}.
- **Estados**: loading (skeleton), vazio (EmptyState).

Detalhe Laboratório — /labs/{id} · acesso R: A,P,T,C,S,CD

Página: pages/laboratorios/LaboratorioDetalhe.tsx · Layout: AppShell + PageHeader + secções

```
┌─ Lab. Química 1  [Química] ──────────────────┐
│ Descrição: ...                               │
├─ Materiais deste lab ────────────────────────┤
│ [tabela mini: nome · qtd · estado]   →       │
├─ Agendamentos futuros ───────────────────────┤
│ [lista: data · hora · actividade]            │
└──────────────────────────────────────────────┘
```

- **Componentes**: PageHeader (nome + Badge tipo); secção materiais (Table mini, link para /materiais/{id}); secção agendamentos futuros (lista).
- **Estados**: loading; vazio por secção (EmptyState).

Cursos — /cursos · acesso CRUD: A · R: P,T,C,S,CD

Página: pages/cursos/CursosList.tsx · Layout: AppShell + PageHeader + Table

```
┌─ Cursos ──────────────────────────────── [+ Novo] ┐
│ Departamento │ Nome          │ Abrev. │ ··· │
│ DET          │ Eng. Informát. │ EI     │ ✎ │
```

```
│ DCSA        │ Gestão        │ GES    │✎ │
└──────────────────────────────────────────┘
```

- **Componentes**: Table (departamento, nome, abreviação), Badge (departamento). "Novo" → CursoUpsertModal.
- **Estados**: loading, vazio, erro.

Disciplinas — /disciplinas · acesso CRUD: A · R: P,T,C,S,CD

Página: pages/cursos/DisciplinasList.tsx · Layout: AppShell + PageHeader + Table

```
┌─ Disciplinas ──────────────────────────── [+ Nova] ┐
│ Nome                    │ ···    │
│ Química Orgânica I          │ ✎     │
│ Física Aplicada             │ ✎     │
├─ Associação Curso ↔ Disciplina ──────────────────┤
│ Curso ▾ · Disciplina ▾ · Semestre · [+]   │
└──────────────────────────────────────────┘
```

- **Componentes**: Table (disciplinas); secção de associação curso/disciplina/semestre. "Nova" → DisciplinasUpsertModal.
- **Estados**: loading, vazio, erro.

Estudantes — /estudantes · acesso CRUD: A,P,C,S,CD · R: T

Página: pages/estudantes/EstudantesList.tsx · Layout: AppShell + PageHeader + FiltersTable + Table

```
┌─ Estudantes ──────────────────────────── [+ Novo] ┐
│ 🔍[nome] [curso ▾]            [Reset]  │
│ Matrícula │ Nome       │ Curso      │ ··· │
│ 20210001  │ Carlos M.  │ Eng. Quím. │ ✎  │
└──────────────────────────────────────────┘
```

- **Componentes**: FiltersTable (nome, curso), Table (matrícula, nome, curso). "Novo" → EstudanteUpsertModal.
- **Estados**: loading, vazio, erro.

Actividades — /actividades · acesso CRUD: A,P,C,S,CD · R: T

Página: pages/actividades/ActividadesList.tsx · Layout: AppShell + PageHeader + FiltersTable + Table

```
┌─ Actividades ──────────────────────────── [+ Nova] ┐
│ [tipo ▾] [estado ▾] [lab ▾]     [Reset]  │
│ Nome        │ Tipo   │ Lab   │ Estado    │ │
│ Aula Quím.  │ Aula   │ LQ1   │🟡Pendente │→│
```

```
│ Visita X    │ Visita │ LF    │🟢Aprovado │→│
└──────────────────────────────────────────────┘
```

- **Componentes**: FiltersTable (tipo, estado, laboratório), Table, Badge de estado colorido (🟡 pendente, 🔵 aprovado_dlab, 🟢 aprovado_supervisor, 🔴 rejeitado) via formatEstado.
- **Ação primária**: "Nova" → ActividadeUpsertModal (multistep):
  1. **Dados base**: nome, laboratório, tipo, nº participantes, precisa_assistente (toggle), observações.
  2. **Detalhes por tipo** (condicional): Aula (disciplina via curso_disciplina_id) · Visita (nome do visitante, instituição, telefone, e-mail) · Projeto/Estágio (título, descrição, datas macro, aluno se estágio).
  3. **Agendamentos**: DateTimePicker repetível (adicionar vários blocos de data/hora).
  4. **Materiais** (opcional): tabela de materiais do lab escolhido + quantidade estimada por sessão.
- **Navegação**: linha → /actividades/{id}.
- **Estados**: loading (skeleton), vazio (EmptyState), erro (Alert).

Detalhe Actividade — /actividades/{id}

Página: pages/actividades/ActividadeDetalhe.tsx · Layout: AppShell + PageHeader + secções

```
┌─ Aula Química Orgânica  [🟢 Aprovado] ─┐
│ Lab: LQ1 · Prof: João · Particip.: 25  │
├─ Detalhes (tipo: Aula) ────────────────┤
│ Disciplina: Química Orgânica I         │
├─ Agendamentos ─────────────────────────┤
│ 01/06 09:00–12:00   [Prof ✓] [Técn ☐]  │
│ 08/06 09:00–12:00   [Prof ☐] [Técn ☐]  │
├─ Técnico & Assistente ─────────────────┤
│ Técnico: Ana (validador)               │
│ Assistente: — (não requerido)          │
├─ Materiais solicitados ────────────────┤
│ Ácido clorídrico · 200ml               │
├─ Documento (projeto/estágio) ──────────┤
│ [EmptyState: "documento ainda não      │
│  submetido"] [FileUpload]              │
├─ Histórico de aprovações ──────────────┤
│ DLab: aprovado (parecer...) · 28/05    │
│ Supervisor: aprovado · 29/05           │
└────────────────────────────────────────┘
```

- **Componentes**: PageHeader (nome + Badge estado); secções condicionais ao tipo (aula/visita/projeto/estágio); lista de agendamentos com toggle realizado (RF14); secção Técnico & Assistente (papéis de actividade_tecnico); materiais solicitados; FileUpload condicional (projeto/estágio sem anexo → EmptyState "documento ainda não submetido"); histórico de aprovações.
- **Estados**: loading; secções vazias com EmptyState.

Aprovações — /aprovacoes · acesso CRUD: A,C,S · R: CD

Página: pages/aprovacoes/AprovacoesList.tsx · Layout: AppShell + PageHeader + Table

```
┌─ Fila de Aprovação (Coordenador DLab) ──┐
│ Actividade   │ Tipo │ Prof │ Data    │ │
│ Aula Quím.   │ Aula │ João │ 01/06   │→│
└──────────────────────────────────────────┘
```

- **Fila adaptativa**: se o utilizador é **C**, mostra atividades pendente; se é **S**, mostra aprovado_dlab.
- **Componentes**: Table. Navegação: linha → /aprovacoes/{id}.
- **Estados**: loading, vazio (EmptyState "Nada para aprovar"), erro.

Detalhe Aprovação — /aprovacoes/{id} · acesso CRUD: A,C,S

Página: pages/aprovacoes/AprovacaoDetalhe.tsx · Layout: AppShell + secções read-only + painel de decisão

```
┌─ Aprovação: Aula Química ──────────────────┐
│ [dados da actividade]                      │
│ [agendamentos propostos]                   │
│ [materiais solicitados]                    │
│ precisa_assistente: Sim                    │
├─ Decisão ──────────────────────────────────┤
│ Comentário/Parecer [_________________]     │
│ (Supervisor) Atribuir técnico [Ana ▾]      │
│ (se assistente) Assistente    [Rui ▾]      │
│    [ Rejeitar ]    [ Aprovar ]             │
└────────────────────────────────────────────┘
```

- **Componentes**: pacote completo read-only (atividade + agendamentos + materiais); Textarea (comentário/parecer obrigatório); na etapa Supervisor, seletores para atribuir Técnico (validador, obrigatório) e Assistente (se precisa_assistente); Button Aprovar/Rejeitar + ConfirmDialog.
- **Regra**: emitir voto cria uma Aprovacao e muda o estado da atividade (aprovado_dlab / aprovado_supervisor / rejeitado).
- **Estados**: loading; submitting (botões desativados + Spinner); erro (Alert).

Calendário — /calendario · acesso: todos

Página: pages/calendario/Calendario.tsx · Layout: AppShell + PageHeader + seletor + grelha mensal

```
┌─ Calendário · [Lab ▾] [Junho 2026 ◂▸] ──┐
│  Seg  Ter  Qua  Qui  Sex  Sáb  Dom      │
│   1    2    3    4    5    6    7       │
│ [Aula]    [Vis]        [choque ⚠]      │
│   8 ...                                 │
└──────────────────────────────────────────┘
```

- **Componentes**: seletor de laboratório e mês; grelha mensal. Só agendamentos de atividades aprovado_supervisor (RF12). Bloco clicável → /actividades/{id}. Alerta visual de choque (RF13).
- **Estados**: loading (grelha skeleton), vazio (mês sem ocupação), erro.

Materiais — /materiais · acesso CRUD: A,T,C,S,CD · R: P

Página: pages/materiais/MateriaisList.tsx · Layout: AppShell + PageHeader + FiltersTable + Table

```
┌─ Materiais ──────────────────────────────── [+ Novo] ┐
│ [lab ▾] [categoria ▾] [estado ▾]  [Reset]  │
│ Nome     │ Cat.   │ Qtd │ Mín │ Estado │ ··· │
│ HCl      │ Comp.  │ 🔴2 │  5  │ Disp.  │ ± → │
│ Béquer   │ Vidr.  │ 40  │ 10  │ Disp.  │ ± → │
└──────────────────────────────────────────────────┘
```

- **Componentes**: FiltersTable (lab, categoria, estado), Table, StockAlertBadge (🔴 quando quantidade ≤ quantidade_minima, RF18). Ação ± → HistoricoMaterialUpsertModal (ajuste manual de stock). "Novo" → MaterialUpsertModal (stock inicial entra como 1ª movimentação).
- **Regra**: coluna Qtd é calculada (SUM do histórico), nunca editável diretamente (RF17).
- **Navegação**: linha → /materiais/{id}.
- **Estados**: loading, vazio, erro. Alertas de stock em tempo real via NotificacoesContext.

Histórico de Materiais — /materiais/historico · acesso CRUD: A,T,S,CD

Página: pages/materiais/MateriaisHistoricoList.tsx · Layout: AppShell + PageHeader + FiltersTable + Table

```
┌─ Histórico de Movimentações ──────────────────┐
│ [material ▾] [motivo ▾] [data]  [Reset]  │
│ Material │ User │ Δ   │ Motivo   │ Data  │
│ HCl      │ Ana  │ -3  │ Consumo  │ 01/06 │
│ HCl      │ Ana  │ +10 │ Compra   │ 15/05 │
└──────────────────────────────────────────────┘
```

- **Componentes**: FiltersTable (material, motivo, data), Table global de movimentações (material, utilizador, +/- quantidade, motivo, data, atividade se houver). Registo de nova movimentação via HistoricoMaterialUpsertModal.
- **Estados**: loading, vazio, erro.

Detalhe Material — /materiais/{id}

Página: pages/materiais/MaterialDetalhe.tsx · Layout: AppShell + PageHeader + cartão de info + Table do histórico

```
┌─ Ácido Clorídrico (HCl) ──────────────────────┐
│ Lab: LQ1 · Categoria: Composto          │
│ Stock atual: 2 (⚠ ≤ mín 5) · Un: ml     │
```

```
├─ Histórico deste material ──────────────────┤
│ -3 Consumo · Ana · 01/06                    │
│ +10 Compra · Ana · 15/05                    │
└──────────────────────────────────────────────┘
```

- **Componentes**: cartão de info (lab, categoria, stock atual = SUM histórico, unidade, estado, StockAlertBadge se aplicável); Table do histórico de movimentações do material.
- **Estados**: loading, vazio (sem movimentações), erro.

Relatórios — /relatorios · acesso CRUD: A,T · R: C,S,CD

Página: pages/relatorios/RelatoriosList.tsx · Layout: AppShell + PageHeader + Table

```
┌─ Relatórios ──────────────────────────── [+ Novo] ┐
│ Lab     │ Mês/Ano │ Criado por │ ···      │
│ LQ1     │ 06/2026 │ Ana        │ →        │
└──────────────────────────────────────────────────┘
```

- **Componentes**: Table (laboratório, mês/ano, criado por). "Novo" → RelatorioUpsertModal (mês, ano, laboratório). Navegação: linha → /relatorios/{id}.
- **Estados**: loading, vazio, erro.

Detalhe Relatório — /relatorios/{id} · acesso R: A,T,C,S,CD

Página: pages/relatorios/RelatorioDetalhe.tsx · Layout: AppShell + PageHeader + render de dados_json

```
┌─ Relatório LQ1 · Junho 2026 ──────────────┐
│ ┌──────────┐ ┌──────────┐ ┌──────────┐    │
│ │Activid.  │ │Realiz.   │ │Materiais │    │
│ │  14      │ │  12      │ │ baixados │    │
│ └──────────┘ └──────────┘ └──────────┘    │
│ [gráfico de barras: actividades/tipo]      │
│                  [Exportar PDF]            │
└──────────────────────────────────────────────┘
```

- **Componentes**: cards de contagem (StatCard), gráfico (recharts) a partir de dados_json, botão exportar PDF.
- **Estados**: loading, erro (JSON inválido → Alert).

3.3. SERVICES

Cada service encapsula chamadas ao axios (api.ts, com interceptor JWT) e mapeia para os DTOs. Os services listados são exatamente os da estrutura de arquivos original; os endpoints das atividades especializadas (aulas, visitas, projetos, estágios) e sub-recursos (agendamentos, técnicos, materiais da atividade) são acomodados nos services existentes para manter coerência com a estrutura sem criar ficheiros novos.

- **auth.service.ts** → login()
- **utilizadores.service.ts** → list, get, create, update, resetPassword, remove
- **laboratorios.service.ts** → list, get, create, update, remove
- **cursos.service.ts** → list, get, create, update, remove · disciplinas e curso-disciplinas (listDisciplinas, createDisciplina, updateDisciplina, removeDisciplina, listCursoDisciplinas, createCursoDisciplina, updateCursoDisciplina, removeCursoDisciplina) · estudantes (listEstudantes, getEstudante, createEstudante, updateEstudante, removeEstudante)
- **actividades.service.ts** → list, get, create, update, remove · especializações e sub-recursos: getAula, upsertAula, getVisita, upsertVisita, getProjecto, upsertProjecto, submeterDocumentoProjecto, getEstagio, upsertEstagio, submeterDocumentoEstagio, listTecnicos, addTecnico, removeTecnico, listMateriais, addMaterial, updateMaterial, removeMaterial
- **aprovacoes.service.ts** → listFila, get, listByActividade, create
- **agendamentos.service.ts** → listByActividade, list, create, update, concluir, remove
- **materiais.service.ts** → list, get, create, update, remove
- **movimentacoes.service.ts** → listAll, listByMaterial, create
- **relatorios.service.ts** → list, get, create, exportarPdf

Nota de coerência: calendario é servido por agendamentos.service.ts (list com filtro laboratorio_id/mes/ano, só aprovado_supervisor), alinhado com o hook useCalendario.

---

4. BACKEND — NODE.JS

Stack Backend

- **express** (framework HTTP)
- **mysql2** + **prisma** (ORM com tipagem e migrations) sobre **MySQL**
- **zod** (validação de inputs, partilhável com o front)
- **jsonwebtoken** (JWT) + refresh tokens
- **argon2** (hash de senhas)
- **helmet** (headers de segurança)
- **express-rate-limit** (proteção brute-force)
- **cors** (configurado por origem)
- **nanoid** (códigos de confirmação de presença; IDs não previsíveis)
- **multer** (upload de anexos PDF)
- **pino** (logs estruturados, leve)
- **socket.io** (alertas de stock em tempo real)
- **node-cron** (relatório mensal automático; checagem de stock mínimo)
- **pdfkit** (geração de PDF dos relatórios; leve, sem Chromium)
- **nodemailer** (alertas por e-mail via SMTP)
- **dotenv** + **envalid** (variáveis de ambiente validadas)

4.1. ESTRUTURA DE ARQUIVOS

```
src/
├── config/
│   ├── env.ts              # envalid — validação de .env
│   ├── db.ts               # ligação MySQL (Prisma client)
│   └── socket.ts           # init socket.io
│
├── middleware/
```

```
│   ├── auth.middleware.ts          # verifica JWT
│   ├── rbac.middleware.ts          # valida utilizador_tipo por rota (RF02)
│   ├── validate.middleware.ts       # aplica schema zod ao request
│   ├── error.middleware.ts         # handler global de erros
│   └── upload.middleware.ts         # multer (PDF)
│
├── modules/
│   ├── auth/
│   │   ├── auth.controller.ts
│   │   ├── auth.service.ts
│   │   ├── auth.routes.ts
│   │   └── auth.schema.ts          # zod: LoginRequest
│   ├── utilizadores/
│   │   ├── utilizadores.controller.ts
│   │   ├── utilizadores.service.ts
│   │   ├── utilizadores.routes.ts
│   │   └── utilizadores.schema.ts
│   ├── cursos/
│   ├── disciplinas/
│   ├── curso-disciplinas/
│   ├── estudantes/
│   ├── laboratorios/
│   ├── materiais/
│   ├── historico-materiais/
│   ├── actividades/
│   ├── aulas/
│   ├── visitas/
│   ├── projectos/
│   ├── estagios/
│   ├── actividade-tecnico/
│   ├── actividade-materiais/
│   ├── agendamentos/
│   ├── aprovacoes/
│   ├── calendario/
│   └── relatorios/
│     # cada módulo segue o padrão: .controller.ts / .service.ts / .routes.ts / .schema.ts
│
├── jobs/
│   ├── relatorioMensal.job.ts       # node-cron: gera relatório de fecho de mês (RF19)
│   └── stockMinimo.job.ts           # checa stock e emite alertas (socket/email) (RF18)
│
├── utils/
│   ├── stock.util.ts               # SUM das movimentações → quantidade (RF17)
│   ├── codigoPresenca.util.ts       # nanoid para confirmação de presença (RF14)
│   ├── pdf.util.ts                 # pdfkit — relatórios
│   └── mailer.util.ts               # nodemailer — alertas
│
├── prisma/
│   ├── schema.prisma
│   └── migrations/
│
├── app.ts                          # express + middlewares globais (helmet, cors, rate-limit)
```

```
└── server.ts                    # http + socket.io listen
```

4.2. BD

Convenções

1. **Soft Delete**: o campo activo (boolean, true = activo) é desativado em vez de apagar o registo.
2. **BaseEntity** (herança/mixin) — todas as tabelas contêm:
   - id integer [pk, increment]
   - criado_em timestamp
   - actualizado_em timestamp
   - activo boolean [default: true]
3. **materiais.quantidade** é **desnormalizado** (cache do SUM(historico_materiais.quantidade_movimentada)), nunca editado diretamente (RF17). É atualizado pelo serviço/trigger a cada movimentação registada.

Enums

```
Enum laboratorio_tipo { quimica, fisica, outro }
Enum departamento_tipo { DET, DCSA, GEO, outro }
Enum utilizador_tipo { admin, professor, tecnico, coordenador_dlab, supervisor, chefe_departamento }
Enum actividade_tipo { aula, visita, projecto, estagio }
Enum actividade_estado { pendente, aprovado_dlab, aprovado_supervisor, rejeitado }   // desnormalização por performance
Enum aprovacao_etapa { dlab, supervisor }
Enum aprovacao_decisao { aprovado, rejeitado }
Enum material_categoria { equipamento, composto, vidraria, consumivel }
Enum material_estado { disponivel, em_uso, manutencao, esgotado }
Enum movimentacao_motivo { consumo_actividade, quebra_acidente, compra_stock, ajuste_inventario, outro }
Enum tecnico_tipo { validador, assistente }
```

Tabelas Base (Utilizadores e Académico)

```
Table utilizadores {
  id integer [pk, increment]
  nome varchar
  email varchar [unique]
  senha_hash varchar
  tipo utilizador_tipo
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table cursos {
  id integer [pk, increment]
  departamento departamento_tipo
  nome varchar
  abreviacao varchar
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
```

```
}

Table estudantes {
  id integer [pk, increment]   // número de matrícula
  nome varchar
  curso_id integer [ref: > cursos.id]
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table disciplinas {
  id integer [pk, increment]
  nome varchar
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table curso_disciplinas {
  id integer [pk, increment]
  curso_id integer [ref: > cursos.id]
  disciplina_id integer [ref: > disciplinas.id]
  semestre integer
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
```

Tabelas de Infraestrutura e Recursos

```
Table laboratorios {
  id integer [pk, increment]
  nome varchar
  tipo laboratorio_tipo
  descricao text
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table materiais {
  id integer [pk, increment]
  laboratorio_id integer [ref: > laboratorios.id]
  nome varchar
  categoria material_categoria
  quantidade integer          // desnormalizado: SUM(historico_materiais.quantidade_movimentada) — RF17
  quantidade_minima integer
  unidade varchar
  estado material_estado
  atualizado_em timestamp
  criado_em timestamp
```

```
  activo boolean [default: true]
}

Table historico_materiais {
  id integer [pk, increment]
  material_id integer [ref: > materiais.id]
  utilizador_id integer [ref: > utilizadores.id]        // quem registou a ação (Professor ou Técnico)
  actividade_id integer [ref: > actividades.id, null]   // opcional (NULL se for acidente/compra)
  quantidade_movimentada integer                        // positivos entrada (+5), negativos saída (-2)
  motivo movimentacao_motivo
  descricao text                                        // justificação detalhada
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
```

Tabelas de Fluxo de Trabalho e Atividades

```
Table actividades {
  id integer [pk, increment]
  nome varchar
  utilizador_id integer [ref: > utilizadores.id]
  laboratorio_id integer [ref: > laboratorios.id]
  tipo actividade_tipo
  estado actividade_estado [default: 'pendente']
  observacoes text
  num_participantes int [default: 1]
  precisa_assistente boolean
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table actividade_tecnico {
  id integer [pk, increment]
  actividade_id integer [ref: > actividades.id]
  utilizador_id integer [ref: > utilizadores.id]
  papel tecnico_tipo          // validador (obrigatório) | assistente (opcional)
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table actividade_materiais {
  id integer [pk, increment]
  actividade_id integer [ref: > actividades.id]
  material_id integer [ref: > materiais.id]
  quantidade_estimada integer
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
```

```
Table aprovacoes {
  id integer [pk, increment]
  actividade_id integer [ref: > actividades.id]
  aprovador_id integer [ref: > utilizadores.id]
  etapa aprovacao_etapa
  decisao aprovacao_decisao
  comentario text
  decidido_em timestamp
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table agendamentos {
  id integer [pk, increment]
  actividade_id integer [ref: > actividades.id]
  hora_inicio datetime        // Ex: 2026-06-01 09:00:00
  hora_fim datetime           // Ex: 2026-06-01 12:00:00
  confirmado_professor_em datetime?
  confirmado_tecnico_em datetime?
  realizado boolean [default: false]  // true só quando ambos confirmarem
  realizado boolean [default: false]
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
```

Tabelas de Especialização de Atividades

```
Table aulas {
  id integer [pk, increment]
  actividade_id integer [ref: - actividades.id]
  curso_disciplina_id integer [ref: > curso_disciplinas.id]
  tema varchar
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
// NOTA: campo precisa_tecnico REMOVIDO — o controlo de técnico/assistente
// vive em actividade_tecnico + actividades.precisa_assistente.

Table visitas {
  id integer [pk, increment]
  actividade_id integer [ref: - actividades.id]
  nome_visitante varchar [not null]
  instituicao varchar
  telefone varchar
  email varchar
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
```

```
}

Table projectos {
  id integer [pk, increment]
  actividade_id integer [ref: - actividades.id]
  responsavel_id integer [ref: > utilizadores.id]
  titulo varchar
  descricao text
  data_inicio date
  data_fim date
  anexo_path varchar
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}

Table estagios {
  id integer [pk, increment]
  actividade_id integer [ref: - actividades.id]
  responsavel_id integer [ref: > utilizadores.id]
  estudante_id integer [ref: > estudantes.id]
  data_inicio date
  data_fim date
  anexo_path varchar
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
```

Tabelas de Auditoria e Eventos

```
Table relatorios {
  id integer [pk, increment]
  laboratorio_id integer [ref: > laboratorios.id]
  criado_por integer [ref: > utilizadores.id]
  mes integer
  ano integer
  dados_json text
  criado_em timestamp
  actualizado_em timestamp
  activo boolean [default: true]
}
// NOTA: total_actividades removido — a contagem é feita por COUNT em runtime
// para evitar mismatch se uma atividade for apagada.
```

4.3. MODELS

As entidades do domínio (Prisma models em schema.prisma) espelham as tabelas acima, todas com os campos de BaseEntity (id, criado_em, actualizado_em, activo).

Relações:

- actividades **1:1** com cada tabela de especialização: aulas, visitas, projectos, estagios (uma atividade materializa-se num único tipo).
- actividades **1:N** com agendamentos, actividade_materiais, actividade_tecnico, aprovacoes.
- materiais **1:N** com historico_materiais (o stock é derivado desta relação).
- cursos **1:N** com estudantes e curso_disciplinas; disciplinas **1:N** com curso_disciplinas.
- laboratorios **1:N** com materiais, actividades, relatorios.
- utilizadores referenciado por actividades (autor), aprovacoes (aprovador), actividade_tecnico (técnico/assistente), historico_materiais (autor do movimento), projectos/estagios (responsável), relatorios (criador).

4.4. DTOs

Utilizadores

- **UserGet** { id, nome, email, tipo, criado_em, actualizado_em }
- **UserUpsert** { id?, nome, email, senha?, tipo } — id não usado na criação; senha não usada na atualização
- **LoginRequest** { email, senha }
- **LoginResponse** { UserGet, token }
- **ResetPassword** { id, nova_senha }

Cursos

- **CursoGet** { id, departamento, nome, abreviacao, criado_em, actualizado_em }
- **CursoUpsert** { id?, departamento, nome, abreviacao }

Disciplinas

- **DisciplinaGet** { id, nome, criado_em, actualizado_em }
- **DisciplinaUpsert** { id?, nome }

Curso e Disciplinas

- **CursoDisciplinaGet** { id, curso_id, curso_nome, disciplina_id, disciplina_nome, semestre, criado_em, actualizado_em }
- **CursoDisciplinaUpsert** { id?, curso_id, disciplina_id, semestre }

Estudantes

- **EstudanteGet** { id, nome, curso_id, curso_nome, criado_em, actualizado_em }
- **EstudanteUpsert** { id?, nome, curso_id }

Laboratórios

- **LabGet** { id, nome, tipo, descricao, criado_em, actualizado_em }
- **LabUpsert** { id?, nome, tipo, descricao }

Materiais

- **MaterialGet** { id, laboratorio_id, laboratorio_nome, nome, categoria, quantidade, quantidade_minima, unidade, estado, criado_em, actualizado_em }
- **MaterialUpsert** { id?, laboratorio_id, nome, categoria, quantidade_minima, unidade, estado } — sem quantidade; stock inicial enviado como 1ª movimentação

Histórico de Materiais

- **HistoricoMaterialGet** { id, material_id, material_nome, utilizador_id, utilizador_nome, actividade_id?, actividade_nome?, quantidade_movimentada, motivo, descricao, criado_em, actualizado_em } — actividade_id/actividade_nome omitidos se NULL
- **HistoricoMaterialUpsert** { id?, material_id, utilizador_id, actividade_id?, quantidade_movimentada, motivo, descricao }

Actividades

- **ActividadeGet** { id, nome, utilizador_id, utilizador_nome, laboratorio_id, laboratorio_nome, tipo, estado, num_participantes, precisa_assistente, observacoes, criado_em, actualizado_em }
- **ActividadeUpsert** { id?, nome, utilizador_id, laboratorio_id, num_participantes, observacoes, precisa_assistente, tipo }

Actividade e Técnico

- **ActividadeTecnicoGet** { id, actividade_id, actividade_nome, utilizador_id, utilizador_nome, papel, criado_em, actualizado_em }
- **ActividadeTecnicoUpsert** { id?, actividade_id, utilizador_id, papel }

Actividade e Materiais

- **ActividadeMaterialGet** { id, actividade_id, material_id, material_nome, quantidade_estimada, criado_em, actualizado_em }
- **ActividadeMaterialUpsert** { id?, actividade_id, material_id, quantidade_estimada }

Aprovações

- **AprovacaoGet** { id, actividade_id, actividade_nome, aprovador_id, aprovador_nome, etapa, decisao, comentario, decidido_em, criado_em, actualizado_em }
- **AprovacaoCreate** (registos imutáveis) { actividade_id, etapa, decisao, comentario? }

Agendamentos

- **AgendamentoGet** { id, actividade_id, hora_inicio, hora_fim, realizado, criado_em, actualizado_em }
- **AgendamentoUpsert** { id?, actividade_id, hora_inicio, hora_fim }
- **AgendamentoConfirmarProfessor { id }**
- **AgendamentoConfirmarTecnico { id }**

Aulas

- **AulaGet** { id, actividade_id, actividade_nome, curso_disciplina_id, curso_disciplina_nome, tema, criado_em, actualizado_em }
- **AulaUpsert** { id?, actividade_id, curso_disciplina_id, tema }

Visitas

- **VisitaGet** { id, actividade_id, actividade_nome, nome_visitante, instituicao?, telefone, email, criado_em, actualizado_em } — instituicao omitida se NULL
- **VisitaUpsert** { id?, actividade_id, nome_visitante, instituicao?, telefone, email }

Projectos

- **ProjectoGet** { id, actividade_id, actividade_nome, responsavel_id, responsavel_nome, titulo, descricao, data_inicio, data_fim, anexo_path?, criado_em, actualizado_em } — anexo_path NULL → "documento ainda não submetido"
- **ProjectoUpsert** { id?, actividade_id, responsavel_id, titulo, descricao, data_inicio, data_fim }
- **ProjectoSubmeterDocumento** { id, anexo_path }

Estágios

- **EstagioGet** { id, actividade_id, actividade_nome, responsavel_id, responsavel_nome, estudante_id, estudante_nome, data_inicio, data_fim, anexo_path?, criado_em, actualizado_em }
- **EstagioUpsert** { id?, actividade_id, responsavel_id, estudante_id, data_inicio, data_fim }
- **EstagioSubmeterDocumento** { id, anexo_path }

Relatórios

- **RelatorioGet** { id, laboratorio_id, laboratorio_nome, criado_por, criado_por_nome, mes, ano, dados_json, criado_em, actualizado_em }
- **RelatorioCreate** { laboratorio_id, mes, ano, dados_json }

4.5. ENDPOINTS

Formato: **Método, chamada, request, response, acesso, descrição.**

1. User /user

Método	Chamada	Request	Response	Acesso	Descrição
POST	/user/login	LoginRequest	LoginResponse	Todos	Login para aceder à plataforma
GET	/user	—	UserGet[]	A	Listar utilizadores
GET	/user/{id}	—	UserGet	A	Obter utilizador
POST	/user	UserUpsert	UserGet	A	Criar utilizador
PUT	/user/{id}	UserUpsert	UserGet	A (auto-edição limitada a nome/senha via /perfil)	Atualizar utilizador
PUT	/user/reset-password	ResetPassword	—	A	Repor senha
DELETE	/user/{id}	—	—	A	Soft delete

2. Cursos /cursos

Método	Chamada	Request	Response	Acesso	Descrição

GET	/cursos	—	CursoGet[]	Todos	Listar cursos
GET	/cursos/{id}	—	CursoGet	Todos	Obter curso
POST	/cursos	CursoUpsert	CursoGet	A	Criar curso
PUT	/cursos/{id}	CursoUpsert	CursoGet	A	Atualizar curso
DELETE	/cursos/{id}	—	—	A	Soft delete

3. Disciplinas /disciplinas

Método	Chamada	Request	Response	Acesso	Descrição
GET	/disciplinas	—	DisciplinaGet[]	Todos	Listar disciplinas
GET	/disciplinas/{id}	—	DisciplinaGet	Todos	Obter disciplina
POST	/disciplinas	DisciplinaUpsert	DisciplinaGet	A	Criar disciplina
PUT	/disciplinas/{id}	DisciplinaUpsert	DisciplinaGet	A	Atualizar disciplina
DELETE	/disciplinas/{id}	—	—	A	Soft delete

4. Curso-Disciplinas /curso-disciplinas

Método	Chamada	Request	Response	Acesso	Descrição
GET	/curso-disciplinas	—	CursoDisciplinaGet[]	Todos	Listar (filtro ?curso_id=)
GET	/curso-disciplinas/{id}	—	CursoDisciplinaGet	Todos	Obter associação
POST	/curso-disciplinas	CursoDisciplinaUpsert	CursoDisciplinaGet	A	Associar disciplina a curso
PUT	/curso-disciplinas/{id}	CursoDisciplinaUpsert	CursoDisciplinaGet	A	Atualizar associação
DELETE	/curso-disciplinas/{id}	—	—	A	Soft delete

5. Estudantes /estudantes

Método	Chamada	Request	Response	Acesso	Descrição
GET	/estudantes	—	EstudanteGet[]	Todos	Listar estudantes
GET	/estudantes/{id}	—	EstudanteGet	Todos	Obter estudante

POST	/estudantes	EstudanteUpsert	EstudanteGet	A,P,C,S,CD	Criar estudante
PUT	/estudantes/{id}	EstudanteUpsert	EstudanteGet	A,P,C,S,CD	Atualizar estudante
DELETE	/estudantes/{id}	—	—	A,P,C,S,CD	Soft delete

6. Laboratórios /labs

Método	Chamada	Request	Response	Acesso	Descrição
GET	/labs	—	LabGet[]	Todos	Listar laboratórios
GET	/labs/{id}	—	LabGet	Todos	Obter laboratório
POST	/labs	LabUpsert	LabGet	A	Criar laboratório
PUT	/labs/{id}	LabUpsert	LabGet	A	Atualizar laboratório
DELETE	/labs/{id}	—	—	A	Soft delete

7. Materiais /materiais

Método	Chamada	Request	Response	Acesso	Descrição
GET	/materiais	—	MaterialGet[]	Todos	Listar (filtro ?laboratorio_id=)
GET	/materiais/{id}	—	MaterialGet	Todos	Obter (quantidade = SUM do histórico)
POST	/materiais	MaterialUpsert	MaterialGet	A,T,C,S,CD	Criar material (+ 1ª movimentação de stock inicial)
PUT	/materiais/{id}	MaterialUpsert	MaterialGet	A,T,C,S,CD	Atualizar (sem quantidade)
DELETE	/materiais/{id}	—	—	A,T,C,S,CD	Soft delete

8. Histórico de Materiais /materiais/historico

Método	Chamada	Request	Response	Acesso	Descrição
GET	/materiais/historico	—	HistoricoMaterialGet[]	Todos	Listar todas as movimentações
GET	/materiais/{id}/historico	—	HistoricoMaterialGet[]	Todos	Histórico de um material

POST	/materiais/historico	HistoricoMaterialUpsert	HistoricoMaterialGet	A,T,S,CD	Registar movimentação (ajuste manual)

9. Actividades /actividades

Método	Chamada	Request	Response	Acesso	Descrição
GET	/actividades	—	ActividadeGet[]	Todos	Listar (filtros tipo/estado/lab)
GET	/actividades/{id}	—	ActividadeGet	Todos	Obter atividade
POST	/actividades	ActividadeUpsert	ActividadeGet	A,P,C,S,CD	Criar atividade
PUT	/actividades/{id}	ActividadeUpsert	ActividadeGet	A,P,C,S,CD	Atualizar atividade
DELETE	/actividades/{id}	—	—	A,P,C,S,CD	Soft delete

10. Aulas /aulas

Método	Chamada	Request	Response	Acesso	Descrição
GET	/aulas/{id}	—	AulaGet	Todos	Obter aula
GET	/actividades/{id}/aula	—	AulaGet	Todos	Aula de uma atividade
POST	/aulas	AulaUpsert	AulaGet	A,P,C,S,CD	Criar detalhe de aula
PUT	/aulas/{id}	AulaUpsert	AulaGet	A,P,C,S,CD	Atualizar aula

11. Visitas /visitas

Método	Chamada	Request	Response	Acesso	Descrição
GET	/visitas/{id}	—	VisitaGet	Todos	Obter visita
GET	/actividades/{id}/visita	—	VisitaGet	Todos	Visita de uma atividade
POST	/visitas	VisitaUpsert	VisitaGet	A,P,C,S,CD	Criar detalhe de visita
PUT	/visitas/{id}	VisitaUpsert	VisitaGet	A,P,C,S,CD	Atualizar visita

12. Projectos /projectos

Método	Chamada	Request	Response	Acesso	Descrição

GET	/projectos/{id}	—	ProjectoGet	Todos	Obter projeto
GET	/actividades/{id}/projecto	—	ProjectoGet	Todos	Projeto de uma atividade
POST	/projectos	ProjectoUpsert	ProjectoGet	A,P,C,S,CD	Criar detalhe de projeto
PUT	/projectos/{id}	ProjectoUpsert	ProjectoGet	A,P,C,S,CD	Atualizar projeto
PUT	/projectos/{id}/documento	ProjectoSubmeterDocumento	ProjectoGet	A,P,C,S,CD	Submeter anexo (pós-conclusão)

13. Estágios /estagios

Método	Chamada	Request	Response	Acesso	Descrição
GET	/estagios/{id}	—	EstagioGet	Todos	Obter estágio
GET	/actividades/{id}/estagio	—	EstagioGet	Todos	Estágio de uma atividade
POST	/estagios	EstagioUpsert	EstagioGet	A,P,C,S,CD	Criar detalhe de estágio
PUT	/estagios/{id}	EstagioUpsert	EstagioGet	A,P,C,S,CD	Atualizar estágio
PUT	/estagios/{id}/documento	EstagioSubmeterDocumento	EstagioGet	A,P,C,S,CD	Submeter anexo (pós-conclusão)

14. Actividade-Técnico /actividade-tecnico

Método	Chamada	Request	Response	Acesso	Descrição
GET	/actividades/{id}/tecnicos	—	ActividadeTecnicoGet[]	Todos	Técnicos/assistentes da atividade

POST	/actividade-tecnico	ActividadeTecnicoUpsert	ActividadeTecnicoGet	A,C,S	Atribuir técnico (validador) ou assistente
DELETE	/actividade-tecnico/{id}	—	—	A,C,S	Remover atribuição

15. Actividade-Materiais /actividade-materiais

Método	Chamada	Request	Response	Acesso	Descrição
GET	/actividades/{id}/materiais	—	ActividadeMaterialGet[]	Todos	Materiais pedidos na atividade
POST	/actividade-materiais	ActividadeMaterialUpsert	ActividadeMaterialGet	A,P,C,S,CD	Adicionar pedido de material
PUT	/actividade-materiais/{id}	ActividadeMaterialUpsert	ActividadeMaterialGet	A,P,C,S,CD	Atualizar quantidade estimada
DELETE	/actividade-materiais/{id}	—	—	A,P,C,S,CD	Remover pedido

16. Agendamentos /agendamentos

Método	Chamada	Request	Response	Acesso	Descrição
GET	/agendamentos	—	AgendamentoGet[]	Todos	Listar (filtro ?laboratorio_id=&mes=)
GET	/actividades/{id}/agendamentos	—	AgendamentoGet[]	Todos	Agendamentos de uma atividade
POST	/agendamentos	AgendamentoUpsert	AgendamentoGet	A,P,C,S,CD	Criar (valida choque — RF13)
PUT	/agendamentos/{id}	AgendamentoUpsert	AgendamentoGet	A,P,C,S,CD	Atualizar
PUT	/agendamentos/{id}/concluir	AgendamentoConcluido	AgendamentoGet	T,A	Marcar realizado via código (baixa stock — RF14)

PUT	/agendamentos/{id}/confirmar-professor	AgendamentoConfirmarProfessor	AgendamentoGet	A,P	Professor marcou como concluído
PUT	/agendamentos/{id}/confirmar-tecnico	AgendamentoConfirmarTecnico	AgendamentoGet	T,A	Técnico marcou como concluído
DELETE	/agendamentos/{id}	—	—	A,P,C,S,CD	Soft delete

17. Aprovações /aprovacoes

Método	Chamada	Request	Response	Acesso	Descrição
GET	/aprovacoes	—	AprovacaoGet[]	A,C,S,CD	Fila adaptativa (C: pendente; S: aprovado_dlab)
GET	/aprovacoes/{id}	—	AprovacaoGet	A,C,S,CD	Detalhe da aprovação
GET	/actividades/{id}/aprovacoes	—	AprovacaoGet[]	Todos	Histórico de aprovações da atividade
POST	/aprovacoes	AprovacaoCreate	AprovacaoGet	A,C,S	Emitir voto (muda o estado da atividade — RF09/RF10/RF11)

18. Calendário /calendario

Método	Chamada	Request	Response	Acesso	Descrição
GET	/calendario	—	AgendamentoGet[]	Todos	Ocupação mensal (só aprovado_supervisor; filtro ?laboratorio_id=&mes=&ano=) — RF12

19. Relatórios /relatorios

Método	Chamada	Request	Response	Acesso	Descrição
GET	/relatorios	—	RelatorioGet[]	A,T,C,S,CD	Listar relatórios
GET	/relatorios/{id}	—	RelatorioGet	A,T,C,S,CD	Obter relatório

POST	/relatorios	RelatorioCreate	RelatorioGet	A,T	Gerar (COUNT de agendamentos realizados — RF19)
GET	/relatorios/{id}/pdf	—	PDF (stream)	A,T,C,S,CD	Exportar relatório em PDF

RESUMO DE DECISÕES E COERÊNCIA

- **Técnico obrigatório vs Assistente opcional**: toda atividade tem um técnico (validador) que gera/valida o código de presença; o assistente é opcional e sinalizado por actividades.precisa_assistente. Ambos ligados via actividade_tecnico. A atribuição ocorre na etapa do Supervisor.
- **aulas.precisa_tecnico**: removido (redundante com o modelo acima).
- **Stock (materiais.quantidade)**: sempre derivado do SUM de historico_materiais; nunca editável diretamente; stock inicial entra como 1ª movimentação (RF17).
- **Mudança de estado da atividade**: ocorre exclusivamente em POST /aprovacoes, evitando inconsistências.
- **Baixa automática de stock**: ocorre em PUT /agendamentos/{id}/confirmar-tecnico (Passo 5).
- **Soft delete**: campo activo boolean (true = activo) em toda a BaseEntity.
- **relatorios.total_actividades**: removido; contagem por COUNT em runtime.
- **Backend**: Node.js + Express + Prisma sobre MySQL, com bibliotecas open-source/leves.
- **Estrutura de ficheiros do front**: mantida fiel ao original; os services acomodam os endpoints especializados sem criar ficheiros novos.